

Agile SRS (Product Requirement Backlog)

for

FinTech Mobile App — "CashPilot"

(Personal Expense Manager)

Version: Sprint 1.0
Product Owner: [Name]
Release Date: TBD (Continuous Delivery)

Agile Software Requirements Specification

1. Product Vision Statement

To become the #1 personal finance app in Southeast Asia by using AI to automatically categorize spending and provide hyper-personalized savings tips, all within a 5-second daily check-in.

2. Epics & User Stories (Functional Requirements)

The requirements are broken down into Epics (large bodies of work) and User Stories (specific functionalities) with defined Acceptance Criteria (AC) to mark them as "Done".

Epic 1: Smart Onboarding & Bank Linking

Goal: Get the user onboarded and their financial data imported in under 2 minutes.

Story ID	User Story	Priority	Acceptance Criteria (AC)
ST-101	As a new user, I want to sign up using my mobile number & OTP so that I don't have to remember a password.	High (Must)	AC 1: User enters a valid 10-digit mobile number. AC 2: System sends a 6-digit OTP via SMS within 5 seconds. AC 3: OTP expires after 60 seconds. AC 4: Upon correct OTP, a JWT token is generated and the Home Screen loads.
ST-102	As a user, I want to securely link my bank account via Plaid API so that transactions are pulled automatically.	High (Must)	AC 1: User selects their bank from a searchable list of 50+ banks. AC 2: System redirects to the bank's OAuth login page (no credentials are stored in CashPilot). AC 3: Upon success, system fetches the last 90 days of transactions. AC 4: A 'Linked' badge appears on the bank card in the UI.
ST-103	As a user, I want to manually add a cash transaction if I pay with physical money.	Medium (Should)	AC 1: User clicks the '+' FAB (Floating Action Button). AC 2: User can input Amount, Description, and Category. AC 3: The transaction appears instantly in the Recent Activity list without needing a page refresh (WebSocket push).

Epic 2: AI Categorization & Budgeting

Goal: Use machine learning to remove the friction of manual expense tracking.

Story ID	User Story	Priority	Acceptance Criteria (AC)
ST-201	As a user, I want the AI to automatically tag my transactions (e.g., "Starbucks" → "Food & Dining") so I don't have to do it manually.	High (Must)	AC 1: Within 30 seconds of a transaction sync, the AI model assigns a primary category and sub-category. AC 2: Confidence score below 70% prompts the user to 'Confirm or Edit' the category. AC 3: User corrections are fed back to the AI model for retraining (data pipeline).
ST-202	As a user, I want to set a monthly budget for categories like "Eating Out" and receive alerts when I am at 80% of my limit.	Medium (Should)	AC 1: User navigates to Budget tab and sets a limit in USD. AC 2: System displays a circular progress bar showing 'Spent / Total'. AC 3: When spending hits 80%, a push notification is sent to the device. AC 4: When spending hits 100%, the category tile turns red.
ST-203	As a user, I want to see a "Weekly Summary" report so I can review my habits quickly.	Low (Could)	AC 1: Every Monday at 9:00 AM, a rich card appears in the app. AC 2: The card shows Total Spent, Top Category, and Money Saved vs. Last Week.

Epic 3: Savings & Goals

Goal: Gamify savings to encourage better financial health.

Story ID	User Story	Priority	Acceptance Criteria (AC)
ST-301	As a user, I want to create a "Goal" (e.g., "Save \$1000 for a new Phone") and track my progress.	High (Must)	AC 1: User enters Goal Name, Target Amount, and Target Date. AC 2: System calculates the 'Monthly Auto-save' amount required. AC 3: The dashboard shows a 'Rainbow' progress bar with a percentage. AC 4: If the user falls behind the monthly schedule, a notification suggests reducing expenses.
ST-302	As a user, I want to enable "Round-Ups" where spare change from purchases goes into my savings vault.	Low (Could)	AC 1: A toggle switch in Settings turns Round-Ups ON/OFF. AC 2: If a transaction is \$4.50, system rounds up to \$5.00 (\$0.50 moved to Vault). AC 3: Vault balance is displayed separately from the main cash balance.

3. Technical Architecture & Non-Functional Specs

Instead of placing these in a separate chapter, Agile teams often define these as "Constraints" or "Engineering Standards" that apply to the entire backlog.

Category	Requirement ID	Detail / Specification
Platform	TECH-001	Mobile: Minimum iOS 14 / Android 10. Backend: AWS (ECS Fargate) for containerized microservices.
Performance	TECH-002	Cold Start: App must launch and show the dashboard within 2.5 seconds on a 4G network. Synchronization: Background sync must occur every 6 hours to pull new bank transactions without killing the battery.
Security	TECH-003	Data Encryption: Sensitive financial data must be encrypted at the application layer (AES-256-GCM) before being sent to the server. Biometrics: Users can optionally enable FaceID/TouchID to unlock the app.
Scalability	TECH-004	The backend must handle 10,000 concurrent API requests during 'Payday' (1st of the month) without latency degradation.
Analytics	TECH-005	All user interactions (button clicks, screen views) must be logged to Mixpanel to track feature adoption and funnel drop-offs.
Localization	TECH-006	The app must support currencies (USD, EUR, GBP) and date formats (MM/DD vs DD/MM). Default falls back to the device's regional settings.

4. UI/UX Design Guidelines (Behavioral Requirements)

Guideline ID	Requirement
UX-01	Dark Mode Support: All screens must render correctly in both Light and Dark themes. Assets must be provided for both.
UX-02	Error Handling: Instead of technical error codes, the UI must show human-readable snackbar messages (e.g., 'Oops! We can't reach your bank right now. Try again in 5 minutes.').
UX-03	Animations: Micro-interactions (like clicking the '+' button) must have haptic feedback on supported devices and a 200ms 'ripple' animation.

5. Glossary of Terms (Team Alignment)

- Plaid: The third-party middleware used to connect to banking institutions.
- FAB: Floating Action Button (the primary action button on the screen).
- Round-Up: The process of rounding a transaction up to the nearest dollar and saving the difference.
- JWT: JSON Web Token used for maintaining user sessions securely.

End of Document